---

title: "KPI Intelligence-to-Action Engine" subtitle: "Complete Project Guide - Architecture, Mechanics, and Usage" author: "BusinessIntelligence.ai Round 2 Prototype" date: "" geometry: margin=1in fontsize: 10.5pt toc: true toc-depth: 2

1. What this project is

This is a working prototype of a "KPI intelligence-to-action engine" — a system that watches business metrics (KPIs), notices when one moves in a meaningful way, figures out *why* using the right analytical tool for the job, explains it differently depending on who's asking, recommends what to do about it, knows when it isn't confident enough to say anything, and tracks what all of that costs to run.

The single most important design decision in the whole codebase, and the thing worth understanding before anything else:

> ***The LLM never computes a number.*** *Every KPI value, every driver contribution, every confidence score is produced by ordinary code — pandas arithmetic, a named statistical test, or a lookup table — before any AI model is called. The model's only job is to turn a finished, structured summary into readable prose for a specific audience.*

This matters because the hackathon brief explicitly warns against treating an LLM as "the source of quantitative truth." Most teams will let the model see raw numbers and trust it to reason about them correctly — that's fragile and unverifiable. Here, the model is architecturally incapable of inventing a number, because it's never shown anything except a dictionary of already-computed values.

---

2. The problem this solves, restated simply

Imagine you're on a sales team and revenue dropped 6% last week. Someone needs to answer: *is this normal noise or a real problem, what caused it, who should know, and what should they do?*

In most companies, answering that requires:

- Pulling data from three different systems that don't talk to each other
- Someone with statistics knowledge deciding if 6% is "a lot" for this particular metric
- Someone digging through spreadsheets to guess whether it was price, volume, a competitor, or bad luck
- Writing that up differently for an executive (who wants one sentence) versus an analyst (who wants the full breakdown)
- Deciding whether to trust the explanation at all, or admit "we don't have enough data to say"

This engine automates that whole chain, end to end, for a small but realistic set of KPIs and data sources.

---

3. The three data sources (and why they're different grains)

Real companies don't have one clean database. They have a sales system that updates daily, a marketing platform that reports weekly, and an operations/supply system that updates monthly. Reconciling these different "grains" (time resolutions) is a real engineering problem, and the prototype simulates it on purpose rather than skip past it.

File	Grain	Simulates
`sales_daily.csv`	one row per day, per region, per product category	the transactional sales system
`marketing_spend_weekly.csv`	one row per week, per channel (not split by region)	a marketing platform export
`ops_monthly.csv`	one row per month, per region, per category	a slow-moving supply/ops system

`data/generate_data.py` creates these with `numpy`'s random number generator (seeded, so it's reproducible), but it also **hand-engineers specific weeks** to contain known scenarios, so the engine has real patterns to discover rather than pure noise:

- **East region, `existing_A` category, final week:** price is deliberately raised 12%, and unit sales are made to drop in response — this is the "clean, explainable multi-driver movement" scenario.
- **West region, `existing_B` category, second-to-last week:** a stockout is injected (units drop sharply) — this shows a driver coming from the slow, monthly ops data rather than the fast sales data.
- **`new_launch` category:** only exists for the last 2 weeks of the 12-week window — this is the sparse-history scenario, where the engine won't have enough data to be confident about anything.

- **West region, `existing_B`, final week:** price ticks up *and* units partially rebound from the prior week's stockout at the same time, while marketing spend moves in a way that produces a correlation pointing the opposite direction from what actually happened — this is the "watch out for a misleading signal" scenario.

---

4. The KPI contract — the governed semantic layer

Before any code touches the data, `contracts/kpi_contracts.yaml` declares what each KPI actually *means*. This is the file that answers "reconcile heterogeneous sources" and "governed KPI semantics" from the brief.

For every KPI it defines:

- **`formula`** — e.g. revenue is `sum(units * price)`, in plain English, not a SQL string the code parses (that's a reasonable next step, not something this prototype does — see Section 11)
- **`source_tables`** — which raw file(s) it comes from
- **`native_grain` / `analysis_grain`** — e.g. daily data reconciled to weekly analysis
- **`materiality_threshold_pct`** — how big a % move has to be before it's "material" for this specific KPI (a 5% swing in average price is a big deal; a 5% swing in units sold might not be)
- **`candidate_drivers`** — which factors are even worth checking for this KPI
- **`access_roles`** — which personas are allowed to see this KPI at all
- **`restricted_dimensions`** — which detail fields get hidden from which roles
- **`lineage`** — a plain-English sentence describing exactly how the number was derived, shown directly in the dashboard next to every value

If you wanted to add a 6th KPI to this system, you would add a block to this YAML file — you would not need to touch any Python code, as long as you also add a matching `elif` branch in `data_loader.py`'s `compute_kpi_series()` function (see Section 5) telling it *how* to compute that KPI's formula.

---

5. Stage-by-stage walkthrough of the pipeline

This section is the heart of the document. Everything below happens, in this exact order, every time `orchestrator.py`'s `run_pipeline()` is called for one KPI + persona + region combination.

5.1 Reconcile grain — `engine/data_loader.py`

`compute_kpi_series("revenue", region="East", category="existing_A")` does:

1. Loads `sales_daily.csv`, filters rows down to East region + existing_A category.
2. Tags every row with the Monday of its week (`week_start`), since sales data is daily but we analyze weekly.
3. Computes `revenue = units * price` for each row, then groups by `week_start` and sums — collapsing 7 daily rows into 1 weekly number.
4. Records `freshness` — the most recent date actually present in the source data, so the dashboard can show "this data is current as of X" next to every finding.

For `marketing_efficiency` (revenue divided by ad spend), this function does something slightly harder: it computes weekly revenue from `sales_daily` *and* pulls weekly spend from `marketing_spend_weekly` (already at the right grain), joins the two on `week_start`, and divides. This is "reconciling heterogeneous sources" made concrete rather than abstract.

5.2 Detect movement — `engine/detection.py`

Two independent statistical checks, no LLM involved:

- **Percent change:** `(latest_value - prior_value) / prior_value * 100`
- **Z-score** (only if at least 5 weeks of history exist): how many standard deviations the latest value is from the trailing average.

A movement is flagged `is_material` if **either** the percent change exceeds the KPI's contract threshold, **or** the z-score's absolute value is at least 2. Using both catches different failure modes — percent change alone would flag a "material" move on a KPI that's always noisy; z-score alone might miss a slow, sustained drift.

This stage also flags `is_sparse_history` if there are fewer periods of data than the contract's `min_periods_for_full_confidence` — this is what triggers the abstention scenario for `new_launch_revenue`.

5.3 Identify and rank drivers — `engine/driver_analysis.py`

Three genuinely different analytical techniques, each tagged with its own method name so nothing is ever attributed to "the model" when it was actually arithmetic:

a) Price-volume-mix (PVM) decomposition. Classic finance/accounting math. Splits the change in revenue between two weeks into how much came from price moving versus how much came from units moving:

```
price_effect  = (this_week_price - last_week_price) * last_week_units
volume_effect = (this_week_units - last_week_units) * last_week_price
              + interaction_term
```

Both are expressed as **percentage points of the prior period's revenue** (not as a share of the net change — see the callout box in Section 9 about why that distinction matters).

b) Pearson correlation. Measures how closely a KPI's weekly values have tracked marketing spend over the last 8 weeks. This is explicitly *never* claimed as causal — every correlation-based driver in the evidence object carries a hardcoded caveat string: *"correlation only - not a causal estimate; confirm with a holdout/geo test before acting."*

c) Rule-based lookup. A plain `if` statement against `ops_monthly.csv`: if this region/category combination has a recorded stockout, emit a driver flag. No magnitude is computed for this one — it's a definite fact ("this happened") but not a quantified contribution.

Ranking. `rank_drivers()` sorts these by **evidence quality first, magnitude second** — decomposition outranks rule-based, which outranks correlation. This was an actual bug caught while building this: initially, a large-but-weak correlation coefficient was outranking the small-but-exact PVM decomposition as the "top driver" simply because its number was bigger. That's exactly the kind of analytical conflation the hackathon brief warns against, and it's why the ranking logic exists as a separate, deliberate step rather than a simple sort by magnitude.

5.4 Score confidence and decide whether to abstain — `engine/confidence.py`

Starts at a score of `1.0` and subtracts points for specific, named reasons:

Condition	Penalty
Fewer periods of history than the KPI needs	-0.35
No z-score could be computed (not enough history)	-0.15
No quantifiable driver was found at all	-0.4
Top two drivers disagree in sign AND are similar in magnitude	-0.3
A correlation-based driver disagrees in direction with a decomposition-based one	-0.15
Only a weak correlation signal is available (no decomposition-grade evidence)	-0.2

If the final score drops below **0.45**, the engine sets `should_abstain = True`. The narrative layer then produces an honest "I'm not confident enough" message instead of guessing at a cause — this is the concrete implementation of the brief's requirement to "communicate uncertainty and abstain when evidence is insufficient or contradictory."

Every subtraction has a plain-English reason attached, and all of the reasons are shown in the dashboard — nothing about this scoring is a black box.

5.5 Build the evidence object — `engine/evidence.py`

This is the architectural boundary of the whole system. `Evidence` is a fixed-shape object containing: the KPI name, the period, the movement numbers, the ranked driver list, the confidence score and its reasons, the data freshness, the lineage string, and the owning role.

Nothing before this point knows an LLM exists. Nothing after this point can see a raw dataframe. That single design choice is what makes "the LLM is not the source of quantitative truth" an enforceable property of the code rather than a hopeful policy statement.

5.6 Enforce entitlements — `engine/security.py`

Two separate checks, both happening *before* the LLM is ever called:

- **Row-level (`check_access`):** does this persona's role appear in the KPI's `access_roles` list? Does the persona's `region_scope` match the region being requested? If a "Regional Manager - West" persona asks about the East region, this returns `allowed=False` and the pipeline stops immediately — the LLM is never even invoked for a denied request.
- **Column-level (`redact_evidence`):** walks the persona's `hidden_dimensions` list (e.g. `["margin", "channel_cost"]`) and replaces matching fields in the evidence object with the literal string `"[redacted]"`, *before* that object is handed to the narrative step.

The important detail: masking happens on structured data, not by asking the LLM nicely not to mention something. A real system that relied on prompting a model to withhold information would be trivially bypassable; this one physically never gives the model the information in the first place.

5.7 Recommend actions — `engine/action_rules.py`

A plain lookup table (`LEVER_LIBRARY`), not LLM generation. For each driver name (`price`, `volume`, `marketing_spend`, `stockouts`, `mix`), there's a hardcoded mapping to a lever, a suggested action, an owning role, and a monitoring plan. The function `recommend_actions()` just fills in the actual contribution percentage from the evidence object into this template. This produces exactly the structure the brief asks for: **driver → controllable lever → action → expected impact → owner → confidence → monitoring plan.**

5.8 Generate the narrative — `llm/` package

This is the *only* stage where a language model is involved, and it happens exactly once per pipeline run. The model receives the finished evidence dictionary (already redacted for the persona) plus the persona's configuration (label, detail level, tone), and its job is purely to phrase it appropriately — nothing more.

The system prompt given to the real model is explicit about this boundary:

> *"Only use numbers and facts present in the evidence JSON. Never invent, estimate, or 'fill in' a figure that isn't there. If evidence.should_abstain is true, do NOT explain a root cause — say plainly that confidence is too low, cite the confidence_reasons given, and suggest what additional evidence would help. Never state a correlation as if it were a proven cause."*

Three provider implementations exist, all behind the same interface (`llm/base_provider.py`):

- **`MockLLMProvider`** — deterministic Python string templates, no API key or internet required at all. Useful for offline development and as a safety net.
- **`GeminiProvider`** — calls Google's Gemini API for real. This is what the prototype actually runs on, since it has a genuinely free tier with no credit card required.
- **`AnthropicProvider`** — calls Claude's API. Written and ready to use, but requires a paid or trial-credited account; not the primary path for this submission since it wasn't available to us in time.
- **`FallbackLLMProvider`** — wraps any "primary" provider with a "fallback" provider. Tries the primary first; if it raises an exception after its own internal retries are exhausted, it catches that and uses the fallback instead, clearly labeling the output's `model` field so a fallback response is never mistaken for a real one (e.g. `"mock-narrative-v1 [FALLBACK - primary unavailable: <reason>]"`).

The actual production configuration used for this submission is:

```
llm = FallbackLLMProvider(primary=GeminiProvider(), fallback=MockLLMProvider())
```

This isn't a hack to paper over problems — it's a deliberate demonstration of the brief's requirement that the system "operate within realistic ... latency and scalability constraints." A real deployment cannot assume its LLM provider has 100% uptime, especially on a free tier that gets rate-limited under load (which we hit directly while building this — see Section 10).

5.9 Telemetry — `engine/telemetry.py`

Wraps every one of the stages above in a timer (`with telemetry.time_stage("name"): ...`), and separately records every LLM call's input/output token counts and an estimated dollar cost. The final pipeline result includes a full breakdown: how many milliseconds each stage took, how many model calls were made, total tokens, and total estimated cost. This is what "LLM economics" looks like as actual numbers instead of a bullet point in a slide deck.

5.10 Feedback loop — `engine/feedback.py`

A SQLite table (`output/feedback.db`) records analyst feedback: was this alert a real finding or a false alarm? If a KPI accumulates 2 or more "false alarm" reports, `suggested_threshold_adjustment()` proposes raising that KPI's materiality threshold by 25% on the *next* run. Deliberately, this is a **suggestion the engine surfaces, never something it applies automatically** — a human has to look at the suggestion and decide. This is a legible, if simple, implementation of "learns from analyst and business-user feedback."

---

6. What ties it all together — `engine/orchestrator.py`

This one file calls everything above, in order, for a single request:

```
check_access
  -> load_and_reconcile_grain
  -> detect_movement
  -> driver_analysis
  -> confidence_scoring
  -> build evidence object
  -> redact for persona
  -> action_rules
  -> llm_narrative_synthesis   (exactly once, at the very end)
```

If you want to see, in one place, exactly which parts of the system are deterministic code and which single step is the LLM, this file is where to look.

---

7. The demo scenarios — `run_scenarios.py`

This script calls the orchestrator 7 times with specific inputs chosen to hit every item in the hackathon brief's "minimum prototype expectations" checklist:

1. **`multi_driver_revenue__exec`**, **`__regional_manager_west`**, **`__sales_analyst`** — the East-region price-hike scenario, viewed by three different personas. The `exec` and `sales_analyst` entries look at *identical* underlying evidence (same region, same category) — this is the clean "two personas, same facts, different narrative" demonstration. The `regional_manager_west` entry is scoped to West instead, since that persona's row-level entitlement wouldn't allow East anyway.
2. **`sparse_history_new_launch`** — the new-product-launch KPI with only 2 weeks of data. Confidence lands around 10%, and the engine abstains.
3. **`low_confidence_abstain`** — the West/`existing_B` scenario with a stockout recovery and a misleading correlation signal, showing calibrated (not binary) confidence.
4. **`security_denied_cross_region`** — `regional_manager_west` attempting to view East-region data, which is blocked before any analysis or LLM call happens.
5. **`feedback_loop_after_2_false_alarms`** — simulates two analysts flagging prior `avg_selling_price` alerts as false alarms, then shows the resulting suggested threshold change.

Each scenario runs through `safe_run()`, which catches any exception (a network timeout that outlasts even the provider's own retries, for example) and records it as an error in that scenario's slot rather than crashing the whole script — so one bad network call can never cost you the other six results.

Results are written to `output/scenarios.json`.

---

8. The dashboard — `build_dashboard.py` and `output/dashboard.html`

`build_dashboard.py` reads `scenarios.json` and bakes it directly into a single, self-contained HTML file — the JSON is embedded in a `<script>` tag, so the dashboard needs no server, no build tool, and no internet connection to open and use. Just double-click `output/dashboard.html` or open it in any browser.

Using it

The dashboard has seven tabs across the top:

- **Alerts feed** — every detected KPI movement from the run, with its percent change and a badge showing whether it was material or the engine abstained.
- **Persona comparison** — the East-region price-hike evidence shown once, next to two different narratives (exec vs. analyst) generated from that same evidence — the clearest single place to point at when explaining requirement 4 (persona narratives).
- **Driver breakdown** — a clickable list of every scenario; selecting one shows its full evidence object (movement, drivers with method badges, confidence bar and reasons, lineage, freshness) plus its generated narrative and recommended actions.
- **Abstention** — the sparse-history scenario, explicitly labeled, with the reasons the engine gave for not naming a cause.
- **Security & entitlements** — shows the denied cross-region request next to an allowed same-region request, plus an explanation of which fields get redacted for which persona and why.
- **Feedback loop** — the false-alarm-driven threshold suggestion.
- **Telemetry & cost** — a table of latency, model calls, token counts, and estimated cost for every scenario, plus totals.

Every number on the page carries a small colored badge showing what produced it: teal for statistics/SQL, amber for rule-based logic, gray for correlation (with its causal-caution wording), and purple for anything the LLM actually wrote. That color coding is the whole thesis of the project made visible at a glance.

---

9. Design choices worth understanding (and being able to defend to judges)

Why percentage points of prior-period revenue, not percentage of net change, for PVM decomposition? When price and volume effects partly offset each other, expressing each as a share of the *net* change produces nonsensical-looking numbers (e.g. "-208%" and "+308%") that are technically correct but read as broken. Expressing each as a percentage of the prior period's base revenue keeps the numbers interpretable and roughly summing to the actual net change.

Why does driver ranking use "evidence quality" tiers instead of just sorting by size? Because a correlation coefficient and a dollar-based decomposition are not the same kind of number, and treating them as comparable is exactly the sloppy reasoning the hackathon brief is testing for. A weak-but-suggestive correlation should never outrank a hard, exact calculation just because its number happens to be numerically larger.

Why is the feedback loop a suggestion, never an automatic write? Because auto-adjusting a business threshold based on a handful of analyst clicks is exactly the kind of unsupervised drift that erodes trust in a system like this. Keeping a human in the loop for that specific action is a deliberate safety choice, not a missing feature.

Why does the LLM layer have a fallback at all? Because a live demo depending on a single external, free-tier API call is fragile — and we hit this directly while building it (see Section 10). A system that gracefully degrades to a clearly-labeled lower-fidelity response, instead of crashing outright, is a more honest reflection of what a production system actually needs to do.

---

10. Real problems we hit while building this (and how they were fixed)

Documenting this because your teammates may hit some of the same things.

"ModuleNotFoundError: No module named 'pandas'" despite installing it. Usually means `pip` and the `python`/`python3` you're running point to two different Python installations on the machine (common on Windows with multiple Python versions installed). Fix: always install with `python -m pip install ...` using the exact same command (`python` vs `python3`) you use to run the scripts, so both go through the same interpreter.

`UnicodeEncodeError: 'charmap' codec can't encode character '\u2192'` when building the dashboard. Windows defaults to the `cp1252` text encoding unless told otherwise, which can't represent the arrow character used in the dashboard. Fixed by explicitly writing the output file as UTF-8 (`out_path.write_text(html, encoding="utf-8")`).

Multi-line `curl` commands silently hanging in Git Bash on Windows. Backslash line-continuations often get mangled when pasted into Git Bash. Fix: put the whole command on a single line, and use `--max-time 20` so a genuine network hang fails loudly with an error instead of hanging forever silently.

`"anthropic-workspace-id is required..."` error from the Anthropic API. Happens when an API key is "identity-linked" (tied to a personal account with access to multiple workspaces) rather than scoped to one workspace. Fixed by either scoping the key to a single workspace at creation time, or passing the workspace ID as an extra header.

`"Your credit balance is too low"` from the Anthropic API. The free trial credit either wasn't granted to that account or was already used. This is why the project ultimately runs on Gemini instead — genuinely free, no card required, no trial-eligibility ambiguity.

Gemini returning `503 Service Unavailable ("high demand")`, sometimes persisting across multiple retries. A real free-tier rate-limiting condition, not a bug in our code. Addressed two ways: (1) `gemini_provider.py` retries automatically on both 5xx errors and outright network timeouts, with increasing backoff between attempts, and (2) `run_scenarios.py` wraps every scenario individually so a persistent failure in one doesn't take down the rest, and (3) the `FallbackLLMProvider` ensures a scenario always produces *something* even if Gemini is completely unavailable for a stretch, with clear labeling of which path actually ran.

---

11. What's deliberately out of scope for this prototype

Worth saying plainly to judges rather than hoping nobody asks:

- **Data is synthetic**, engineered to contain specific known scenarios. Real connectors to actual source systems are a separate, solvable engineering problem — this prototype is about the reasoning architecture, not data plumbing.
- **The contradictory-evidence check is a heuristic**, not a formal statistical test. It reliably catches the obvious case (a correlation pointing the opposite way from a hard decomposition) but isn't a rigorous causal-inference method.
- **KPI formulas are hardcoded per-KPI in Python**, keyed off the contract, rather than parsed generically from the YAML's formula string. A more general system would parse and execute the formula expression itself; this prototype trades that generality for legibility within the build window.
- **The feedback loop is a simple rule-of-thumb**, not a learning algorithm.

---

12. Quick reference — file map

Path	What it does
`contracts/kpi_contracts.yaml`	Governed KPI definitions, thresholds, roles, lineage
`data/generate_data.py`	Creates the 3 synthetic source CSVs with engineered scenarios
`engine/data_loader.py`	Reconciles source grain to weekly analysis grain per KPI
`engine/detection.py`	Materiality detection (z-score, pct-change)
`engine/driver_analysis.py`	PVM decomposition, correlation, rule-based stockout check, ranking
`engine/confidence.py`	Confidence scoring and abstention rule
`engine/evidence.py`	The structured evidence object - the LLM/non-LLM boundary
`engine/security.py`	Personas, row/column-level entitlement enforcement

Path	What it does
`engine/action_rules.py`	Driver -> lever -> action -> owner -> monitoring lookup
`engine/feedback.py`	Feedback capture and threshold-adjustment suggestion
`engine/telemetry.py`	Per-stage latency and per-call token/cost tracking
`engine/orchestrator.py`	Wires every stage together for one pipeline run
`llm/base_provider.py`	The interface every LLM provider implements
`llm/mock_provider.py`	Deterministic template narratives, no API key needed
`llm/gemini_provider.py`	Real Gemini API calls, with retry logic
`llm/anthropic_provider.py`	Real Claude API calls (written, unused in this submission)
`llm/fallback_provider.py`	Wraps a primary + fallback provider pair
`run_scenarios.py`	Runs all 7 required demo scenarios, writes `scenarios.json`
`build_dashboard.py`	Bakes `scenarios.json` into the standalone HTML dashboard
`output/dashboard.html`	The final, self-contained deliverable - open this in a browser

---

13. How to run it, start to finish

```
# from inside the kpi_engine/ folder
python -m pip install pandas numpy pyyaml requests

# 1. Generate the synthetic source data
python data/generate_data.py

# 2. Set your Gemini API key (get one free, no card, at aistudio.google.com)
export GEMINI_API_KEY=your_key_here
#   Windows PowerShell instead: $env:GEMINI_API_KEY="your_key_here"

# 3. Run the full pipeline across all 7 required scenarios
python run_scenarios.py

# 4. Build the dashboard
python build_dashboard.py

# 5. Open the result
#    just double-click output/dashboard.html, or:
open output/dashboard.html          # Mac
start output/dashboard.html         # Windows
xdg-open output/dashboard.html      # Linux
```

If you don't have a Gemini key yet, or don't want to use one, everything still runs end-to-end without it — just leave `run_scenarios.py` using `MockLLMProvider` instead of `GeminiProvider`, and every scenario will produce deterministic template narratives instead of live-generated text. No internet connection is required in that mode at all.